

Marketing Mix Modeling

Portfolio Project Guide — Bayesian MMM with PyMC-Marketing

Sameer · July 2026

Project Overview & Roadmap

Marketing Mix Model (MMM) — Portfolio Project

Goal: Build a Bayesian Marketing Mix Model on real retailer data using PyMC-Marketing, quantify each channel's ROI, and recommend an optimal budget reallocation.

Resume bullet (target):

Built a Bayesian Marketing Mix Model (PyMC-Marketing) on 4 years of weekly retail data across 10 media channels; quantified channel ROI with uncertainty intervals and recommended a budget reallocation projected to lift revenue by X% at constant spend.

Project structure

```
mmm-portfolio-project/
├── README.md           ← you are here (roadmap + setup)
├── LEARNING_GUIDE.md   ← MMM concepts + interview prep
├── requirements.txt
├── data/
│   └── DATA_DICTIONARY.md ← what every column means
└── notebooks/
    ├── 01_data_exploration.ipynb ← load, clean, understand the data
    └── 02_build_mmm.ipynb        ← build, validate, and use the model
```

Setup (one time, ~15 min)

1. Install Python via [Miniconda](#) (recommended — handles PyMC's dependencies cleanly).
2. Open a terminal (macOS: Terminal app) and run:

```
cd ~/Downloads/mmm-portfolio-project
conda create -n mmm python=3.11 -y
conda activate mmm
pip install -r requirements.txt
jupyter lab
```

1. Jupyter opens in your browser. Open `notebooks/01_data_exploration.ipynb` and run cells top-to-bottom with Shift+Enter.

Roadmap (4 weeks, ~5–8 hrs/week)

Week	Milestone	Output
1	Read LEARNING_GUIDE.md; run notebook 01; understand every column	Clean modeling dataset
2	Run notebook 02 through model fitting; understand adstock/saturation/priors	Fitted model + diagnostics
3	ROI analysis, saturation curves, budget optimizer	Channel ROI table + reallocation recommendation
4	Write it up: 1-page case study + README polish; push to GitHub	Portfolio piece + interview story

The dataset

Real (anonymized) US retailer data, 200+ weeks (2014–2018), from the well-known [sibylhe/mmm_stan](#) repository (originally a Kaggle dataset). Notebook 01 downloads it automatically. See `data/DATA_DICTIONARY.md`.

Stack decision (know this for interviews)

Chose **PyMC-Marketing** over Robyn (Meta) and Meridian (Google) because: - Fully Bayesian → ROI estimates come with credible intervals ("Search ROI is $2.1x \pm 0.4$ "), which is what budget decisions need. - Python, transparent model specification — every prior and transform is visible and defensible. - Most-downloaded open-source MMM library; excellent docs. - Robyn = ridge regression + evolutionary hyperparameter search (fast but harder to defend statistically). Meridian = strongest for geo-level data, which this dataset doesn't have.

MMM Learning Guide

MMM Learning Guide

Read this before touching the notebooks. ~30 minutes.

1. The business problem

A CMO spends across TV, search, social, radio, print. Sales happen. The question MMM answers: **how much of each week's sales did each channel cause, and where should the next dollar go?**

Why not just track clicks (attribution)? Because attribution only sees digital, breaks with privacy changes (iOS14, cookie deprecation), and can't measure TV/radio/print. MMM works on aggregate data — no user tracking — which is why demand for MMM skills has surged since 2021.

2. The core model

At its heart, MMM is a regression on weekly data:

$$\text{sales}_t = \text{baseline} + \sum_{\text{channels}} \beta_c \cdot f(\text{spend}_c, t) + \sum \gamma \cdot \text{controls}_t + \text{error}$$

- **baseline** — sales you'd get with zero advertising (brand strength, distribution, habit)
- **controls** — price, promotions, holidays, seasonality, macro factors
- **f()** — the two transforms below. This is what makes it MMM and not just regression.

Transform 1: Adstock (carryover) — ads have an afterglow

Story: you run a pizza shop and spend ₹10,000 on a TV ad on Monday. Some people order that evening. Ramesh sees it Monday but orders Friday. Priya orders two weeks later when friends visit. The ad sits in people's heads and keeps nudging them — that is "carryover": the effect *carries over* into future weeks. "Adstock" = the leftover stock of influence the ad still has.

If the model only compared this week's spend to this week's sales, Ramesh's and Priya's orders would look like they came from nowhere and TV would get too little credit. So we replace "spend this week" with "advertising pressure in the air this week":

$$\text{pressure}_t = \text{fresh spend}_t + \alpha \cdot \text{pressure}_{t-1} \quad (0 < \alpha < 1)$$

Worked example with $\alpha = 0.5$ (memory halves each week): spend ₹10,000 once → pressure is 10,000 / 5,000 / 2,500 / 1,250 over weeks 1–4.

α is a **memory dial**. Search ads: $\alpha \approx 0$ (someone searching "pizza near me" buys *now*). TV brand ads: $\alpha \approx 0.6$ –0.8 (afterglow lasts weeks). The model learns each channel's dial from the data.

Transform 2: Saturation (diminishing returns) — the millionth rupee works less hard

Story: your town has 50,000 people. The first ₹1 lakh of ads reaches people who've never heard of you — lots of new customers. The next lakh mostly reaches people who already saw the ad — fewer new customers. By the fifth lakh, everyone reachable has been reached; extra money buys almost nothing. That's saturation.

So the model fits a curve that rises steeply then flattens. Where a channel sits on its curve — steep part (more budget pays off) or flat part (saturated) — **is literally the "where should the next rupee go" answer.**

Why Bayesian?

- Weekly data is small (200 rows) and channels are correlated → point estimates are unstable.
- Priors let you encode marketing knowledge ("TV decay is probably 0.5–0.8").
- You get uncertainty: "Search ROI is 2.1x, 94% credible interval [1.6, 2.7]" — an honest basis for a budget decision. This is your strongest interview talking point.

3. The workflow

1. **Data prep** — one row per week: sales, spend per channel, controls. (Notebook 01)
2. **Specify model** — choose adstock + saturation forms, priors, seasonality. (Notebook 02)
3. **Fit** — MCMC sampling (NUTS). Takes 10–40 min on a laptop.
4. **Validate** — convergence diagnostics ($r_{\text{hat}} < 1.01$, no divergences), posterior predictive check (does simulated sales look like actual?), out-of-sample fit.
5. **Decompose** — split each week's sales into baseline + per-channel contributions.
6. **ROI** — channel contribution $\div$ channel spend.
7. **Optimize** — reallocate budget along saturation curves to maximize predicted sales.

4. Vocabulary you must own

Term	Meaning
KPI / target	The thing you model (weekly revenue here)
Adstock / carryover	Lagged, decaying effect of past spend
Saturation / Hill curve	Diminishing returns to spend
Baseline	Sales not attributable to media
Contribution	Sales attributed to a channel in the decomposition
ROI / ROAS	Attributed revenue $\div$ spend
mROI (marginal ROI)	Return on the <i>next</i> dollar — what optimization actually uses
Prior / posterior	Belief before / after seeing data (Bayesian)
r_{hat} , divergences	MCMC convergence diagnostics
Calibration	Anchoring MMM with experiment results (lift tests) — mention in interviews

5. Interview questions to prepare for

1. *Why MMM over last-click attribution?* → privacy-proof, covers offline, measures incrementality not correlation of clicks.
2. *Why did you pick PyMC over Robyn/Meridian?* → see README; know all three.
3. *How do you know your model isn't garbage?* → convergence diagnostics, posterior predictive checks, out-of-sample validation, ROI estimates within plausible industry ranges, contributions sum sensibly.
4. *Difference between ROI and marginal ROI?* → average vs derivative of the saturation curve; budget decisions use marginal.
5. *Limitations of MMM?* → correlational (not a randomized experiment), needs 2+ years of data, channel collinearity, can't do fine-grained targeting. Gold standard = MMM calibrated with lift tests.
6. *What if two channels always move together?* → multicollinearity → wide posteriors; fixes: priors, combining channels, calibration experiments.

6. Going deeper (optional, in order)

1. PyMC-Marketing MMM example notebooks — pymc-marketing.io
2. "Bayesian Methods for Media Mix Modeling with Carryover and Shape Effects" (Jin et al., Google, 2017) — the paper behind modern MMM
3. Robyn docs (facebookexperimental.github.io/Robyn) — to speak to the comparison
4. Meridian docs (developers.google.com/meridian) — geo-hierarchical MMM

Data Dictionary

Data Dictionary

Source: [sibylhe/mmm_stan](#) — anonymized US retailer, weekly data Aug 2014 onward (200+ weeks). Notebook 01 downloads `data.csv` automatically.

Column groups

Prefix / column	Meaning
<code>wk_strt_dt</code>	Week start date (the time index)
<code>sales</code>	Weekly sales revenue (\$) — the KPI we model
<code>mdsp_*</code>	Media spend (\$) per channel per week — main predictors
<code>mdip_*</code>	Media impressions per channel (same channels; alternative to spend)
<code>me_*</code>	Macro-economic controls
<code>st_ct</code>	Store count (distribution)
<code>mrkdn_*</code>	Markdown / discount value (promo intensity)
<code>hldy_*</code>	Holiday dummy flags (1 = that week contains the holiday)
<code>seas_*</code>	Seasonality dummy flags

Media channels (the `mdsp_` suffixes)

Suffix	Channel	Our grouping in notebook 01
<code>dm</code>	Direct mail	<code>print_mail</code>
<code>inst</code>	Newspaper insert	<code>print_mail</code>
<code>nsp</code>	Newspaper ad	<code>print_mail</code>
<code>audtr</code>	Traditional radio	<code>audio</code>
<code>auddig</code>	Digital audio	<code>audio</code>
<code>vidtr</code>	Traditional video (TV)	<code>tv</code>
<code>viddig</code>	Digital video (YouTube etc.)	<code>digital_video</code>
<code>so</code>	Social media	<code>social</code>
<code>on</code>	Online display	<code>display</code>
<code>sem</code>	Search engine marketing	<code>search</code>

Why group 10 → 7? With ~200 weekly observations, fewer channels means more data per parameter and less collinearity. Grouping decisions like this are a legitimate modeling choice you should be able to defend.

Key controls we keep

- `me_ics_all` — consumer sentiment index (macro demand)
- `me_gas_dpg` — gas price \$/gallon (consumer spending power)
- `st_ct` — store count
- `mrkdn_valadd_edw`, `mrkdn_pdm` — markdown/promo intensity
- Selected `hldy_*` flags (Black Friday, Christmas, Prime Day, Thanksgiving, July 4th)

Yearly seasonality is handled inside the model with Fourier terms, so we drop `seas_*` dummies.